Track the Game

Architecture and VLM Benchmark Report

GitHub Repo: `https://github.com/Aun-29120/Track-The-Game`

Contents

1 Architecture and Pipeline

This system processes 30-second sports video clips to automatically generate per-player team markers, ball highlights, and dynamic possession indicators. Rather than relying on traditional computer vision heuristics such as edge detection or background subtraction, the visual perception is managed entirely by a Vision-Language Model (VLM). The system utilizes a robust data pipeline to handle video processing and the mathematics required to track players smoothly over time.

The architecture is divided into five distinct stages:

1.1 1. Video Decoding and Normalization

Before inference can occur, the video must be efficiently decompressed and separated into individual frames.

- **Multithreaded Extraction**: The system utilizes `PyAV` to decode the video stream. By enabling multithreading (`stream.thread_type = "AUTO"`), the extraction process is significantly accelerated compared to standard sequential processing.
- **Rotation Correction**: Mobile footage often contains hidden rotation metadata. The pipeline cross-references the frame dimensions using OpenCV to detect and automatically correct any improper rotation.
- **Frame Rate Standardization**: To ensure that the downstream tracking mathematics remain synchronized with actual time, the footage is resampled to exactly 30.0 frames per second using variable frame rate (VFR) timestamp matching.

1.2 2. Spatial Grounding (Synthetic Overlay)

Vision-Language Models do not inherently understand absolute pixel coordinates. To extract precise locations, the system visually grounds the model by providing a reference frame.

- A 40-pixel thick white band is appended to the top (X-axis) and left (Y-axis) edges of the video frame.
- A synthetic ruler is drawn onto these borders, scaling from 0 to 1000, complete with major and minor tick marks.
- By referencing this scale, the VLM can return highly accurate numerical coordinates (e.g., $X = 450, Y = 800$). A translation matrix then converts these normalized units back into exact original screen pixels. This synthetic ruler is removed prior to final video rendering.

1.3 3. VLM Perception and Sparse Detection

Processing every frame of a 30-second video would violate both latency and cost constraints. Therefore, the system evaluates sparse "keyframes" (e.g., one frame every 8 or 16 frames).

- **Image Optimization**: To reduce network payload and API processing time, keyframes are resized using a Lanczos filter so their longest edge is exactly 1280 pixels.
- **Concurrent Processing**: The system dispatches up to 64 frames to the VLM simultaneously using `httpx.AsyncClient`, maximizing throughput without hitting rate limits.

- **Structured Constraints**: The VLM is governed by a strict JSON schema using `pydantic`. It is instructed to classify goalkeepers and referees as "Gray", extract legible jersey numbers, and ignore any individuals standing outside the green pitch boundary.

1.4 4. Temporal Interpolation and Identity Tracking

Because the VLM analyzes each keyframe independently, it lacks temporal context. The system must logically stitch these isolated detections together into continuous tracks.

- **Identity Matching**: To associate the same player across different frames, the system constructs a cost matrix and applies the Hungarian optimization algorithm (`linear_sum_assignment`). Matches are primarily prioritized based on minimum spatial movement.
- **Semantic Identity Signals**: If the VLM detects a matching jersey number across frames, a significant mathematical reward (-1000.0) is applied to guarantee the linkage. Conversely, mismatched team colors incur a penalty (+50.0).
- **Outlier Rejection**: If a player's coordinates shift impossibly far across the screen (exceeding 150 ruler units), the link is rejected to prevent "teleportation" tracking errors.
- **Smooth Interpolation**: For the non-keyframe gaps, the system utilizes Monotonic Spline Interpolation (`PchipInterpolator`). This specific algorithm calculates the precise curve of the player's movement without overshooting or introducing unnatural wobbles.

1.5 5. Hardware-Accelerated Rendering

Once the movement paths are fully calculated, the graphical overlays are drawn directly onto the original, un-ruled video arrays.

- **Marker Generation**: Colored ellipses are drawn at the base of the players' feet using highly efficient OpenCV native rendering (`cv2.ellipse`).
- **Possession Calculation**: The system mathematically determines possession by calculating the Euclidean distance between the ball and all valid players. Any player within a 50-pixel threshold receives an additional white possession indicator ring.

2 How to Run the Project

The pipeline is executed via a central command-line interface.

1. **Environment Setup**: Create a `.env` file in the root directory to define the required API credentials and configuration variables:

```
OPENROUTER_API_KEY="sk-or-v1-..."
VLM_MODEL="google/gemini-3.5-flash-lite"
MAX_CONCURRENT_VLM_CALLS=64
```

2. **Mock Testing (Optional)**: To validate the pipeline logic without incurring API costs, append `USE_MOCK_VLM=true` to the `.env` file.

3. **Execution**: Initiate the pipeline through the terminal:

```
python scripts/run_pipeline.py --in data/clips/clip1.mp4 \
                               --out data/output/clip1_annotated.mp4
```

3 Benchmark Methodology

The system was evaluated against the rigorous constraints outlined in the project brief:

- **Target Latency**: Process a 30-second video in under 15.0 seconds, with an absolute accepted maximum ceiling of 25.0 seconds.
- **Target Cost**: Maintain a total API expenditure strictly under $1.00 per finished video.
- **Hardware Environment**: macOS utilizing Apple Silicon, leveraging hardware-accelerated `VideoToolbox` for the final video encoding process.
- **Model Selection**: `google/gemini-3.5-flash-lite` via the OpenRouter API.

4 Benchmark Results

The following table presents the end-to-end processing performance across five distinct 30-second clips. Clips 1 through 4 utilized a keyframe interval of 8 (analyzing approximately 115 frames per clip). Clip 5 was processed at an interval of 16 (analyzing 57 frames) to evaluate the impact on processing speed and API cost.

Clip	Frames Processed	Keyframes Sent	Interval	Elapsed (s)	Cost ($)
Clip 1	892	112	8	49.7	0.3307
Clip 2	919	115	8	56.8	0.4499
Clip 3	948	119	8	52.8	0.4323
Clip 4	907	114	8	51.7	0.3854
Clip 5	901	57	16	62.2	0.1773

Table 1: End-to-end benchmark results for all 5 clips. The pipeline successfully remained under the $1.00 cost limit for all videos.

4.1 Output Frames

Figure 1: Rendered frames demonstrating tracking accuracy and possession overlays across Clips 1 through 5.

A Appendix: Attempts to Bring Down Latency

The initial iteration of this pipeline required 103.8 seconds to process a single video. A series of deep technical optimizations successfully reduced this execution time by half, though the system remains inherently bottlenecked by network API response times.

- **Hardware Video Encoding**: Initially, drawing the markers and encoding the video consumed 37.8 seconds due to OpenCV's unoptimized `mp4v` CPU encoder. The backend was refactored to stream the image arrays directly into PyAV's `h264_videotoolbox` hardware encoder. Shifting this workload to the GPU reduced the rendering step to just 12.60 seconds.
- **Connection Pool Scaling**: Dispatching 115 network requests simultaneously requires proper management of TCP connections. The pipeline was explicitly configured (`httpx.Limits(max_connections=64)`) to support 64 concurrent open connections, preventing internal system queuing before the requests even left the local network.
- **Asynchronous Threading**: Converting high-resolution arrays into JPEG files is a CPU-bound task that ordinarily blocks the main Python process. By wrapping this image compression task in a background thread (`asyncio.to_thread`), the main event loop was liberated to focus entirely on managing asynchronous network requests.

Latency Conclusion: Despite these software optimizations, the pipeline averages approximately 50 seconds, structurally missing the 25-second maximum target. Because all API requests are dispatched as a single large batch, the total processing time is strictly dictated by the *single slowest* AI response in that batch (tail-latency). Local hardware optimizations cannot bypass this external cloud network bottleneck.

B Appendix: Costs and Interval Ablation

The project guidelines mandate a strict maximum budget of $1.00 per finished video.

- **Interval 8 Performance**: By tracking only every 8th frame, the system analyzes roughly 115 frames per video rather than the full 900. By utilizing the highly economical `gemini-3.5-flash-lite` model, this strategy maintains the total API cost comfortably between $0.33 and $0.45 per video, operating well below the required budget.
- **Interval 16 Performance (Ablation)**: For Clip 5, the keyframe interval was increased to 16, successfully reducing the number of AI requests to 57. Consequently, the cost dropped linearly to $0.1773. However, the total execution time remained high at 62.2 seconds. This confirms that while halving the workload reduces overall cost, it does not lower the speed limit imposed by the slowest individual network request in the batch.